

Solution Architecture

High-level architecture mapped to the uploaded Presentation / Application / Data template

Project	aiblognest	Team ID	6ab4ccd16f290be8a6b460f4
---------	------------	---------	--------------------------

Architecture Flow

Layer / Component	aiblognest implementation
Presentation / Client Layer	Postman / Thunder Client or any REST client consuming the API. No dedicated frontend application was supplied.
API Gateway / Application Entry	Express.js server running on Node.js; API routes expose authentication, blog, and AI operations.
Auth Service	JWT-based authentication with bcryptjs for password hashing; protected routes require valid authentication.
Core Logic Service	Express route/controller logic for user profiles, blog CRUD, AI generation, and summarization; project documentation describes an MVC architecture.
External API Integration	Google Gemini integration through @google/generative-ai for AI blog generation and summarization.
Data / Storage Layer	MongoDB accessed through Mongoose for persistent application data.

Component Description Table

Component Name	Description / Role	Technology Used
Presentation Layer	Sends REST requests and displays API responses during testing or client integration.	Postman / Thunder Client / REST client
API Gateway / Server	Receives HTTP requests, routes them to application logic, and returns responses.	Node.js, Express.js
Authentication	Registers users, authenticates credentials, issues JWTs, and protects routes.	JWT, bcryptjs
Core Logic	Handles blog CRUD and AI-related API operations.	Express.js, MVC-style organization
AI Integration	Sends content-generation/summarization requests to the configured Gemini model.	Google Gemini, @google/generative-ai
Database	Persists users and blog data through an object-document mapping layer.	MongoDB, Mongoose

Architectural Notes

The supplied architecture template includes an API Gateway/Load Balancer and Microservices placeholders. For aiblognest, the documented implementation is a Node.js/Express backend rather than a demonstrated microservices deployment. No production load balancer, cloud-hosting topology, or separate microservice deployment was supplied, so these are not asserted as implemented.

High-level flow: REST Client → Express/Node.js → Authentication / Blog / AI Logic → MongoDB and Gemini integration → API response.